Chapter 2 One-Pager — Game Theory and CFR Basics

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Problem. Chapter 1's machinery assumes a fully observable state and a stationary environment; poker has neither. Two-player zero-sum imperfect-information games need a different solution concept — Nash equilibrium defined over *information sets* — and a different quality measure: exploitability, not reward. Chapter 2 establishes both, plus the algorithm that computes them, because everything after it is either an approximation of CFR (Chapters 3-5), a deliberate departure from the equilibrium CFR produces (Chapters 7-8), or an attempt to say what equilibrium even means once `N > 2` (Chapter 9, 11).

Approach. Vanilla CFR written from scratch on **Kuhn Poker** — 3 cards, 2 players, 12 information sets: small enough to have a closed-form equilibrium *family* (parameterised by `alpha` in `[0, 1/3]`), rich enough to contain bluffing, mixed strategies and indifference. Hand-coded components: the game engine, recursive counterfactual traversal with regret matching and chance sampling, and an exact exploitability evaluator that enumerates all `2^6 = 64` pure strategies — because a per-state "oracle" best response is *not* a best response; the best-responder must commit to one action per information set. Trained for 100,000 iterations, with an OpenSpiel cross-verification script alongside. **All numbers below are measured** (`implementation/step02/models/cfr_results.json`).

Key results (measured).

- *CFR lands inside the analytical equilibrium family, not merely near it.* The recovered bluff parameter is `alpha = 0.1941`, and the family's own internal constraint `P(bet | K) = 3*alpha` holds to **2.6e-4** (measured **0.58247** against the **0.58221** its own alpha implies).
- *Pure decisions converge hard; mixed frequencies carry the O(1/sqrt(T)) tail.* King bets and calls at **>= 0.9999**, Jack folds to a bet at **0.99998**, Queen passes at the root at **0.99993** — while the mixing sits **0.002-0.011** off the closed form (Jack bluff after a pass **0.3403** vs `1/3`; Queen call **0.5387** vs `1/3 + alpha = 0.5274`). The frequencies are the slow part, which is precisely where the residual regret lives.
- *Game value* **-0.0602** measured against the exact **-1/18 = -0.0556** at 100k iterations, reproducing Player 0's structural first-mover disadvantage.
- *The convergence rate is the theoretical one.* Log-log exploitability slope **-0.489** against a predicted **-0.5**.
- *Game value alone is not a validity check.* A strategy that always bluffs the Jack can still average near `-1/18` while being trivially exploitable; only exploitability catches it. That is why exploitability — not reward — is the metric carried forward into Chapters 7-8 and 14.

Thesis connection. Nash is the baseline this thesis exists to leave: Contribution #1 reads a specific opponent in order to justify departing from it, and Contribution #2 bounds how far the departure may go. Concretely, the exact best-response and exploitability code written here becomes the literal oracle reused in Chapters 7-10, and Kuhn remains the smallest testbed where every later claim can be bracketed by an exact answer.

Open questions. Why the *average* strategy converges when the current strategy does not — the intuition (averaging damps the overshoot, as Polyak averaging does) is not a proof. Everything here rests on the two-player zero-sum minimax theorem, and neither CFR's guarantee nor exploitability's meaning survives into N-player or general-sum settings (Chapter 9, 11). And full-tree traversal touches every information set on every iteration, so this exact algorithm is already out of budget one game up — the motivation for Chapter 3's Monte Carlo sampling and Chapter 4's abstraction.